Requerimientos de integración — Tienda Virtual

De: Equipo B (Catálogo y Host App) · **Para:** Equipo A (Búsqueda) y Equipo C (Carrito)
Fecha: 13 de noviembre de 2026
Entrega: martes 17 de noviembre de 2026. Para alcanzar a probar todo junto, pedimos tener los cambios en la rama que usa el `docker-compose` de integración (Búsqueda: `main`; Carrito: `develop`) **el lunes 16 de noviembre**.

Este documento reúne, en un solo lugar, lo que le falta a cada equipo para que la tienda funcione completa dentro del Host App. Los acuerdos de fondo están en el contrato de Catálogo v2.5 y en el contrato del Host App v1.3; aquí solo va lo que hay que hacer, en orden de importancia, y cómo comprobar que quedó.

1. Lo que ya está listo del lado del Equipo B

Pieza	Estado
Host App en el `5050`	Encabezado con el menú **Catálogo · Búsqueda · Carro · Órdenes** y el botón **Iniciar sesión**. Carga cada módulo en un iframe
Login con Keycloak	Realm `tienda` en `http://localhost:8180`. El Host App inicia la sesión y le pasa el token a cada módulo con el mensaje `sesion` (contrato del Host App v1.3, sección 5.1)
«Órdenes»	Entrada del menú que abre `/carrito/pedidos` en el módulo de Carrito
Catálogo	API v2.4 (`referencia`, `devolver-stock`, `items` en el 409, `activo` y `fecha` en los eventos) y v2.5 (la administración pide token). Productos de prueba con ids fijos
Kong único en el `8000`	Rutas `/api/catalogo`, `/api/busqueda` y `/api/carrito`, con CORS para `5050`, `5173`, `5174` y `5175`
`docker-compose` de integración	En teambsoft-hostapp/integracion: los tres backends construidos desde GitHub, sus bases, un RabbitMQ, Kong y Keycloak

Así se ve la tienda cuando todo encaja (es el recorrido de la presentación):

1. El usuario abre `http://localhost:5050` y pulsa **Iniciar sesión** → Keycloak → vuelve con su nombre en el encabezado.
2. En **Catálogo** abre un producto y pulsa **Agregar al carro**.
3. El Host App abre **Carro** en `/carrito?agregar={id}`; Carrito agrega una unidad **con el usuario del login** y muestra el carrito.
4. Paga: Carrito descuenta el stock en Catálogo con su `checkoutId` como `referencia`.
5. En **Órdenes** aparece el pedido de ese mismo usuario.
6. En **Búsqueda**, el producto aparece con el stock actualizado, porque Búsqueda recibió el evento de Catálogo.

Los pasos 1 a 5 ya los probamos de punta a punta el 13 de noviembre, con el backend y el frontend de Carrito modificados con el parche de la sección 2.3. El paso 6 depende de la sección 3.

2. Equipo C — Carrito

2.1 Qué falta

#	Qué	Prioridad	Por qué
C1	Frontend en el puerto `5175` con `strictPort: true`	**Bloquea**	Hoy está en el `5173`, que es el de Catálogo: los dos no pueden correr a la vez. Lo aceptaron el 12 de noviembre (contrato del Host App v1.2)
C2	Al cargar, si la dirección trae `?agregar={id}`, agregar una unidad con `POST /api/carrito/items` y quitar el parámetro (`history.replaceState`)	**Bloquea** «Agregar al carro»	Es el paso 3 del recorrido (contrato del Host App v1.2, sección 9.1)
C3	Si la dirección es `/carrito/pedidos`, abrir directamente «Mis pedidos»	**Bloquea** «Órdenes»	El menú «Órdenes» del Host App lleva ahí
C4	**Recibir la sesión** del Host App: escuchar el mensaje `sesion`, pedirla con `pedir-sesion` y usar ese token en las llamadas. Si no hay sesión, ofrecer **Iniciar sesión** con `iniciar-sesion`	Alta	Sin esto, el carrito usa el token de prueba y la persona que inició sesión no es la que compra ni la que ve sus órdenes
C5	En el backend, **aceptar el token de Keycloak** (RS256, con la llave pública del realm) **además** del token de prueba (HS256 con `JWT_SECRET`)	Alta	El token de C4 lo firma Keycloak, no su `JWT_SECRET`. El de prueba se sigue aceptando: sus pruebas no cambian
C6	Ocultar su encabezado («Demo Carrito de Compras») cuando corre embebido: `window.self !== window.top`	Media	Si no, se ven dos encabezados (contrato del Host App v1.2, requisito 3)
C7	Unir `feature/SCRUM-28-frontend-historial` a `develop`	**Bloquea** C3	El `docker-compose` de integración construye su rama `develop`, y la pantalla de pedidos todavía no está ahí
C8	Actualizar su contrato v1.4	Baja	Su sección 7.1 dice que Catálogo no valida tokens y que cada equipo levanta su Kong, y la 7.2 marca como pendientes la `referencia`, `devolver-stock` e `items` en el 409: los tres ya están en Catálogo v2.4, y desde la v2.5 hay un solo Kong y Keycloak

2.2 Datos para C4 y C5

Dato	Valor
Origen del Host App (de quien se acepta `sesion`)	`http://localhost:5050`
Mensaje que llega	`{ tipo: 'sesion', token, usuario }`; `token: null` cuando no hay sesión o se cerró

Dato	Valor
Mensajes que el módulo puede enviar	`{ tipo: 'pedir-sesion' }` y `{ tipo: 'iniciar-sesion' }`, al origen `http://localhost:5050`
Renovación	El Host App vuelve a mandar `sesion` cada vez que renueva el token (~cada 4 minutos)
Emisor del token (`iss`)	`http://localhost:8180/realms/tienda`
Llaves públicas, dentro de Docker	`http://keycloak:8080/realms/tienda/protocol/openid-connect/certs`
Algoritmo	RS256. El `kid` de la cabecera dice qué llave usar
Usuario	El claim `sub` (un UUID), igual que hoy

El `docker-compose` de integración ya le pasa al backend de Carrito `KEYCLOAK_LLAVES` y `KEYCLOAK_EMISOR`. Si su versión todavía no las usa, las ignora.

2.3 Parche ya probado

Para ahorrarles tiempo, escribimos y probamos los cambios C1 a C6 sobre su repositorio: integracion/carrito-integracion.patch. **No tocamos su repositorio:** lo aplican ustedes, lo revisan y lo suben como cualquier cambio suyo.

Archivo	Qué cambia
`Frontend/vite.config.js`	Puerto `5175` con `strictPort` (C1)
`Frontend/src/hostApp.js` (nuevo)	Todo lo del Host App: `embebido`, recibir la sesión, pedir el login, leer `?agregar=` y saber si se abrió en `/carrito/pedidos`
`Frontend/src/main.js`	Si está embebido, espera la sesión del Host App antes de montar, para que la primera llamada ya salga con el usuario correcto (C4)
`Frontend/src/App.vue`	Oculta el `<h1>` embebido (C6), aviso con **Iniciar sesión** si no hay sesión (C4), abre «Mis pedidos» en `/carrito/pedidos` (C3) y agrega el producto de `?agregar=` antes de mostrar el carrito (C2)
`src/config/env.js`	Variables `KEYCLOAK_LLAVES` y `KEYCLOAK_EMISOR`. Vacías = todo como antes
`src/middlewares/authJwt.js`	Acepta RS256 de Keycloak (descarga la llave una vez y la guarda) y sigue aceptando HS256 con `JWT_SECRET` (C5)
`tests/authKeycloak.test.js` (nuevo)	5 pruebas: token de Keycloak válido, token de prueba, otro emisor, otra llave y token vencido. No necesita Keycloak: genera sus propias llaves

Cómo aplicarlo (después de C7, porque el parche va sobre la pantalla de pedidos):

```
git switch develop
git pull
git switch -c integracion-host-app
git apply ruta/a/carrito-integracion.patch
npm test
```

Qué comprobamos con el parche aplicado:

- `npm test` en su repositorio: **198 pruebas en verde** (172 de Jest, incluidas las 5 nuevas, y 26 de `node --test`).
- Su `Frontend` compila con `npm run build`.
- Con el `docker-compose` de integración, su backend construido desde el parche y los cuatro frontends: el recorrido de la sección 1, pasos 1 a 5, completo. El usuario `cliente` inició sesión, agregó la camiseta desde Catálogo, la compró (el stock bajó de 100 a 99 y Catálogo guardó la referencia `chk_…`) y la vio en «Órdenes».
- Sin sesión, el módulo muestra el aviso y su botón lleva al login del Host App.
- `curl` con el token de Keycloak a `GET /api/carrito` responde `200` con el `sub` del usuario; sin token, `401`.

2.4 Dos observaciones de la revisión

- En `Frontend/src/stores/cartStore.js` dice `e.mensagem` (en portugués) en lugar de `e.mensaje`. Por eso cualquier error del backend se muestra como «Ocurrió un error inesperado» en lugar del mensaje real. No está en el parche: es de ustedes.
- Su backend envía `X-Frame-Options: DENY`. **No hay que quitarlo:** solo aplica a las respuestas de la API, y lo que el Host App carga en el iframe es el frontend, que sirve Vite.

3. Equipo A — Búsqueda

Revisamos su repositorio el 13 de noviembre: el último cambio en `main` es del 5 de noviembre.

3.1 Qué falta

#	Qué	Prioridad	Por qué
A1	**Consumidor de eventos:** enlazar su cola al exchange **`catalogo.eventos`** (hoy `catalogo.events`) con las routing keys **`producto.creado`**, **`producto.actualizado`**, **`producto.desactivado`** y **`producto.reactivado`** (hoy `catalogo.producto.*`)	**Bloquea**	Con los nombres actuales **no les llega ningún evento**: el 12 de noviembre Catálogo publicó 6 y su cola recibió 0. Exchange y routing keys son los del contrato desde la v2.1
A2	Aplicar `producto.reactivado` y **tomar `activo` del evento**, en vez de marcar el producto como activo por su cuenta	**Bloquea**	Un producto desactivado que se edita volvería a aparecer en la búsqueda (contrato de Catálogo, sección 5.1)

#	Qué	Prioridad	Por qué
A3	Frontend en el puerto `5174` con `strictPort: true`	**Bloquea**	Hoy no fija ningún puerto, así que Vite usa el `5173`, el de Catálogo
A4	Frontend llama a su backend **por Kong**: `VITE_API_URL=http://localhost:8000/api/busqueda`	**Bloquea**	Hoy va directo a `http://localhost:3000`, que en el `docker-compose` de integración no está publicado
A5	Ocultar su encabezado («Búsqueda de productos») cuando corre embebido: `window.self !== window.top`	Media	Si no, se ven dos encabezados
A6	*(Opcional)* Al pulsar un resultado, llevar al detalle en Catálogo: `window.parent.postMessage({ tipo: 'navegar', ruta: '/catalogo/' + id }, '*')`	Baja	Une los dos módulos en la presentación: buscar → ver el producto → agregar al carro

El login no les cambia nada: Búsqueda es pública y sus endpoints no piden token.

3.2 Cómo comprobar A1 y A2

Con el `docker-compose` de integración arriba:

1. Abrir la consola de RabbitMQ (`http://localhost:15672`) → *Queues* → `busqueda.catalogo.eventos` debe mostrar los **6 mensajes** de los productos de prueba en *Message rates* al arrancar.
2. Buscar «Teclado» en Búsqueda: **no** debe aparecer, porque el producto `674000000000000000000006` está desactivado.
3. Como administrador, reactivarlo en Catálogo: en unos segundos debe aparecer en Búsqueda.

4. Datos de referencia

Qué	Dónde
Host App	`http://localhost:5050`
Frontends	Catálogo `5173`, Búsqueda `5174`, Carrito `5175`
Kong	`http://localhost:8000/api/catalogo`, `/api/busqueda`, `/api/carrito`
Keycloak	`http://localhost:8180` (consola en `/admin`, usuario y clave `admin` / `admin`)
Usuarios de prueba	`admin` / `admin123` (roles `admin` y `cliente`) y `cliente` / `cliente123` (rol `cliente`)
Token para curl o Postman	`POST http://localhost:8180/realms/tienda/protocol/openid-connect/token` con `grant_type=password`, `client_id=tienda-pruebas`, `username` y `password`
RabbitMQ	Exchange `catalogo.eventos` (topic), routing keys `producto.*`. Consola en `http://localhost:15672`
Productos de prueba	Ids `674000000000000000000001` a `…006` (contrato de Catálogo, sección 10.5)

5. Cómo probar todo junto

```
# En teambsoft-hostapp/integracion, con el .env lleno (las claves se comparten por privado)
docker compose up -d --build
```

Y en cuatro terminales:

Frontend	Comando
Host App	`npm run dev` en `teambsoft-hostapp`
Catálogo	`KONG_URL=http://localhost:8000 npm run dev` en `teambsoft-frontend` (PowerShell: `$env:KONG_URL="http://localhost:8000"; npm run dev`)
Búsqueda	`npm run dev` en `busqueda-frontend`, con `VITE_API_URL` de A4
Carrito	`npm run dev` en `Frontend`

Después, el recorrido de la sección 1. Si algo falla, los logs de cada servicio salen con `docker compose logs -f <servicio>`.

6. Historial de cambios

Fecha	Cambio
2026-11-13	Versión inicial, después de revisar los repositorios de Búsqueda (`main`, 5 de noviembre) y Carrito (`develop` y sus ramas del 12 y 13 de noviembre), y de probar el parche de Carrito de punta a punta